
```
>>> iterableObj = range(3)
>>> iterableObj
range(0, 3)
>>> iteratorObj = iter(iterableObj)
>>> i = next(iteratorObj)
>>> print(i) # body of the for loop
0
>>> i = next(iteratorObj)
>>> print(i) # body of the for loop
1
>>> i = next(iteratorObj)
>>> print(i) # body of the for loop
2
>>> i = next(iteratorObj)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
```

❶ StopIteration

Notice that if you call `next()` after the last item in the iterable has been returned, Python raises a `StopIteration` exception ❶. Instead of crashing your programs with this error message, Python's `for` loops catch this exception to know when they should stop looping.

An iterator can only iterate over the items in an iterable once. This is similar to how you can only use `open()` and `readlines()` to read the contents of a file once before having to reopen the file to read its contents again. If you want to iterate over the iterable again, you must call `iter()` again to create another iterator object. You can create as many iterator objects as you want; each will independently track the next item it should return. Enter the following into the interactive shell to see how this works:

```
>>> iterableObj = list('cat')
>>> iterableObj
['c', 'a', 't']
>>> iteratorObj1 = iter(iterableObj)
>>> iteratorObj2 = iter(iterableObj)
>>> next(iteratorObj1)
'c'
>>> next(iteratorObj1)
'a'
>>> next(iteratorObj2)
'c'
```
